4.6.1 Primary Index

Search Key: Ordered field and key (candidate key)

- I/O cost to access record using primary index with multilevel indexing is (k + 1) blocks.
  Where k is level of indexing.
- For any database relation at most one primary index is possible because, search key must be ordered field.
- Primary index can be either dense or sparse but sparse primary index preferred.

4.6.2 Clustered Index

Search Key: Ordered field and non-key.

- I/O cost:

  $$\text{Single level index blocks} : \left\lceil \frac{\text{No. of distinct values over non-key}}{\text{BF of Index}} \right\rceil$$

  $$\therefore \text{ Access cost} = \left\lceil \log_2(\text{single level index blocks}) \right\rceil + 1$$

- I/O cost to access cluster of record using clustered index with multilevel index is:

  $$k + \underbrace{(\text{one or more block of DB})}_{\text{Until next cluster begins}}$$

- For any DB relation at most one clustering index is possible because search key is ordered field.

> **Note:**
> For any DB relation either primary index or clustering index is possible but not both simultaneously.

4.6.3 Secondary Index [over key]

Search Key : Unordered field and key.

- Secondary index is alternative index to access data records even primary or clustering index already exists.
- Secondary index over key is always dense index.
- I/O cost to access record using secondary index over key with multilevel index: (k + 1) blocks.

4.6.4 Secondary Index [over non-key]

Search Key: Unordered field and non-key.

- I/O cost to access record using secondary index over non-key with multilevel index = [k blocks from k level of index] + [some more DB blocks]

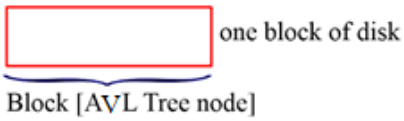
> **Note:**
> Many secondary index's possible for DB file.

4.7 Balanced Search Tree [Height Restricted Search Tree]

- The maximum height of search tree for n distinct keys should not exceed O (log n)
- Worst case search cost to search element is : O (log n)

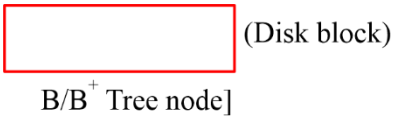
4.7.1 Why B/B+ Tree Index preferred rather than balanced BST/AVL for DB index

(a) AVL: If AVL tree (balanced BST) used for DB index:



- Disadvantage: Each index block with only one key so the number of levels of index is high. (I/O cost to access data is also very high).
- Wastage of disc space, which is allocated for index (only one key stored per block)

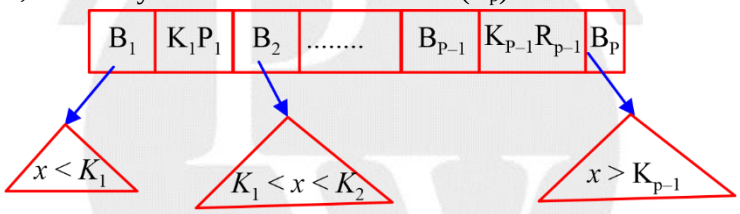
(b) B/B$^+$ Tree:



- The access cost (I/O cost) is less because many keys store in one node (number of levels of index will be less).
- The wastage of disk space is less.

4.7.2 B Tree Definition

- Order P (degree): The maximum possible child printers can be stored in B Tree node.
  Node structure: P child pointer, P – 1 Key and P – 1 Record Pointer (R_p).



$\therefore$ order of node $\Rightarrow$

$P \times$ (size of block pointer) + (P - 1) (size of key field + size of record pointer) $\leq$ Block size

1. Every internal node except root must be atleast $\left\lceil \frac{p}{2} \right\rceil$ block pointer and $\left\lceil \frac{p}{2} \right\rceil$ - 1 keys and at most P block pointer and (P – 1) keys.
2. Root can have at least 2 child/block pointer and 1 keys and at most P block pointer and (p - 1) keys.
3. Every leaf node must be at same level.

- **Advantages:**

B Tree index best suitable for random access of any one record.

Example:

```
SELECT *
FROM R
WHERE A = 15;
```

$\therefore$ Worst case access cost = (k + 1) blocks
Best case access cost = 2 blocks

- **Disadvantages:**

B Tree index not suitable for sequential access of range of records.

4.7.3 B+ Tree

Order P: The maximum possible pointers can be store in B^+ tree node.

1. Node Structure:

- **Leaf node:** (set of (key, R_P) pairs and one block pointer)

K_1R_1	K_2R_2	…….	$K_{P-1}R_{P-1}$	B_P

→Block Pointer {pointer to next leaf}

∴ Order of leaf node = $(P-1)[K + R_P] + 1 * B_P \leq$ Block size.

- **Internal Node:**
  Order of internal node = $P * B_P + (P-1) * K \leq$ Block size.
- B^+ Tree best suitable for random access of any one record and sequential access of range of records.
- I/O cost to access range of 'x' records sequential:

$$K + \frac{x}{\left\lceil \frac{P}{2} \right\rceil - 1} + x = 0(K + x)$$

To reach leaf

No. of times leaf accessed

DB block

Important point 2

If disk block allocation for B and B^+ tree nodes are equal size then

1. Order P of B tree < order P of B^+ Tree.
2. [number of index blocks of B tree index for n keys] ≥ [number of index blocks of B^+ Tree nodes for n keys]
3. I/O cost ≥ I/O cost

Important point 3

If order P of B Tree is equal to order P of B^+ Tree then

1. [number of B Tree node n distinct keys] ≤ [number of B^+ Tree nodes for n distinct keys]
2. [B Tree level for n keys] ≤ [B^+ Tree levels for n keys]

❑❑❑